

Les types de données - Partie 2

Les nombres

Les nombres sont utilisés pour représenter des valeurs numériques. En JavaScript, vous pouvez utiliser des nombres entiers (comme 42) ou des nombres décimaux à virgule (comme 3.14). On parle alors de "int" pour les entiers et de "float" pour les nombres décimaux. Certains langages de programmation font la distinction entre les deux, mais en JavaScript, tous les nombres sont traités de la même manière.

```
let nEntier = 42; // Un nombre entier
let nDecimal = 3.14; // Un nombre décimal

console.log(nEntier); // Affiche: 42
console.log(nDecimal); // Affiche: 3.14
console.log(nEntier + nDecimal); // Affiche: 45.14 (additionne les deux nombres)
```

Opérations arithmétiques (mathématiques) sur les nombres

Vous pouvez effectuer des opérations mathématiques de base sur les nombres, comme l'addition, la soustraction, la multiplication et la division. Voici quelques exemples :

```
let nA = 10;
let nB = 5;

let nAddition = nA + nB; // Addition
let nSoustraction = nA - nB; // Soustraction
let nMultiplication = nA * nB; // Multiplication
let nDivision = nA / nB; // Division

console.log(nAddition); // Affiche: 15
console.log(nSoustraction); // Affiche: 5
console.log(nMultiplication); // Affiche: 50
console.log(nDivision); // Affiche: 2
```

Opérateur +, un même opérateur pour additionner et concaténer

Vous avez peut-être remarqué que l'opérateur + est utilisé à la fois pour additionner des nombres et pour concaténer des chaînes de caractères. En JavaScript, si vous utilisez + avec au moins une chaîne de caractères, il effectuera la concaténation plutôt que l'addition.

Cela peut parfois prêter à confusion, surtout si vous essayez d'additionner des nombres et des chaînes de caractères. Par défaut, JavaScript convertit les nombres en chaînes de caractères si l'un des opérandes est une chaîne.

```
let nNombre = 5;
let sTexte = "10";

let nResultat = nNombre + sTexte; // Ici, JavaScript convertit nNombre en chaîne
et concatène
console.log(nResultat); // Affiche: "510" (une chaîne de caractères)
```

Convertir une chaîne de caractères en nombre avant de faire des opérations mathématiques

Pour éviter ce problème, vous pouvez convertir explicitement une chaîne de caractères en nombre en utilisant la fonction `Number()`. Lorsque vous récupérez des données utilisateur, il est courant de devoir convertir des chaînes en nombres pour effectuer des calculs car les données saisies par l'utilisateur sont presque toujours des chaînes de caractères.

Vous pouvez également utiliser `parseInt()` pour convertir une chaîne en entier ou `parseFloat()` pour convertir une chaîne en nombre décimal mais `Number()` fait les deux

```
let sNombre = "10";
let nNombre = Number(sNombre); // Convertit la chaîne en nombre

let nResultat = nNombre + 5; // Maintenant, l'addition fonctionne correctement
console.log(nResultat); // Affiche: 15 (un nombre)

let sReponse = prompt("Entrez un nombre :"); // Pose une question à l'utilisateur
et stocke la réponse dans une variable
let nReponse = Number(sReponse); // Convertit la réponse en nombre. Remarquez que
nous avons utilisé un préfixe différent pour éviter la confusion avec le texte
let nAddition = nReponse + 100; // Additionne 100 à la réponse de l'utilisateur
console.log(nAddition); // Affiche le résultat correct de l'addition
```

Les valeurs booléennes

Les valeurs booléennes sont utilisées pour représenter deux états possibles : vrai (`true`) ou faux (`false`). Elles sont souvent utilisées dans les conditions pour contrôler l'exécution du programme, par exemple dans les instructions `if`, les boucles, etc.

```
let bEstVrai = true; // Une valeur booléenne vraie
let bEstFaux = false; // Une valeur booléenne fausse
console.log(bEstVrai); // Affiche: true

let nNombre = 10;
let bEstPlusGrandQueCinq = nNombre > 5; // Vérifie si nNombre est plus grand que 5
et stocke le résultat dans une variable booléenne qui sera true ou false
console.log(bEstPlusGrandQueCinq); // Affiche: true (car 10 est bien plus grand
que 5)
```

Opérateurs de comparaison

Dans l'exemple précédent, nous avons utilisé l'opérateur `>` pour comparer deux nombres. Il existe plusieurs opérateurs servant à comparer des valeurs et à retourner des valeurs booléennes:

- `==`: Égal à (vérifie si deux valeurs sont égales, sans tenir compte du type Ex: `5 == "5"` retourne `true` car JavaScript convertit la chaîne en nombre lors de la comparaison)
- `>`: Plus grand que (vérifie si la valeur de gauche est supérieure à celle de droite Ex: `10 > 5` retourne `true`)
- `<`: Plus petit que (vérifie si la valeur de gauche est inférieure à celle de droite Ex: `3 < 5` retourne `true`)
- `>=`: Plus grand ou égal à (vérifie si la valeur de gauche est supérieure ou égale à celle de droite Ex: `5 >= 5` retourne `true`)
- `<=`: Plus petit ou égal à (vérifie si la valeur de gauche est inférieure ou égale à celle de droite Ex: `5 <= 10` retourne `true`)
- `!=`: Différent de (vérifie si deux valeurs ne sont pas égales, sans tenir compte du type Ex: `5 != 6` retourne `true`)

Note: Il existe aussi les opérateurs `===`, `!==` qui sont beaucoup plus stricts. Ils vérifient l'égalité en tenant compte du type de données. Par exemple, `5 === "5"` retourne `false` car le type de données est différent (nombre vs chaîne de caractères). Pour l'instant, nous n'utiliserons pas ces opérateurs mais vous les rencontrerez dans d'autres contextes.

```
let nA = 10;
let nB = 5;

let bEgalite = nA == nB; // Vérifie si nA est égal à nB
console.log(bEgalite); // Affiche: false (car 10 n'est pas égal à 5)

let bPlusGrand = nA > nB; // Vérifie si nA est plus grand que nB
console.log(bPlusGrand); // Affiche: true (car 10 est plus grand que 5)

let bPlusPetit = nA < nB; // Vérifie si nA est plus petit que nB
console.log(bPlusPetit); // Affiche: false (car 10 n'est pas plus petit que 5)

let bPlusGrandOuEgal = nA >= nB; // Vérifie si nA est plus grand ou égal à nB
console.log(bPlusGrandOuEgal); // Affiche: true (car 10 est plus grand que 5)

let bPlusPetitOuEgal = nA <= nB; // Vérifie si nA est plus petit ou égal à nB
console.log(bPlusPetitOuEgal); // Affiche: false (car 10 n'est pas plus petit que 5)

let bDiff = nA != nB; // Vérifie si nA est différent de nB
console.log(bDiff); // Affiche: true (car 10 est différent de 5)
```